

一、专业工具——Git&Github

1、Git基础

什么是Git?

Git是一个分布式版本控制系统，能记录每次代码的改动，方便回溯、协作和防止代码丢失。(即保存代码进度)

版本控制?

- 1、Git会记录每一次代码的变更，可以轻松查看所有历史记录
- 2、Git可以创建独立的分支，在分支上可以自由地进行实验
- 3、Git可以自动合并一个团队中所有人对同一文件的修改，并且给出冲突部分
- 4、通过查看提交记录，可以有效解决bug

4个核心命令：(终端命令行)

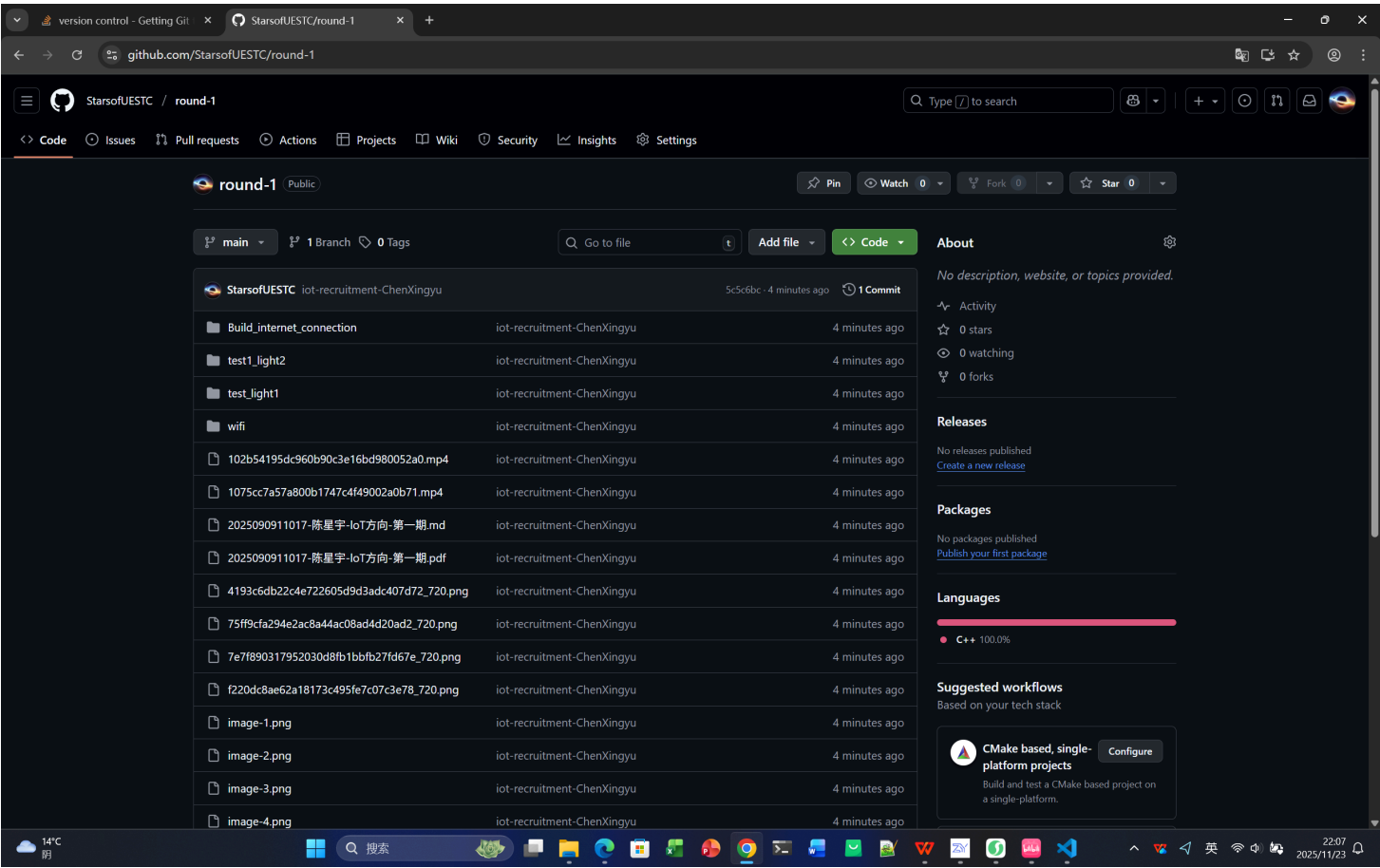
git clone:将远程仓库的代码克隆、复制到本地

git add:将文件改动添加到暂存区

git commit:将暂存区的改动正式提交到本地仓库

git push:推送至远程仓库

2、代码总仓库



建立过程比较轻松，跟B站教程学即可，网卡的时候用了下梯子来进行代理



编写规范的README.md文件：

version control - Getting Git x StarsofUESTC/round-1 x StackEdit

github.com/StarsofUESTC/round-1

README

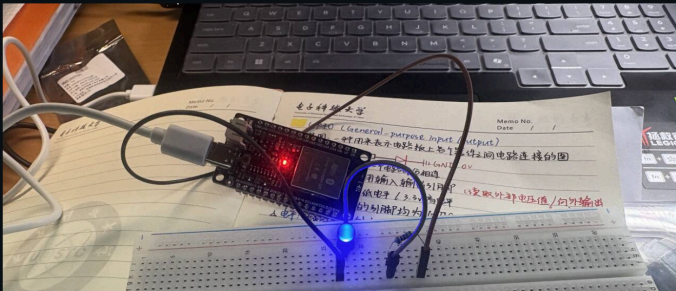
项目：点灯&联网

1、点灯

1.1、使用ESP32开发板和ArduinoIDE开发环境，执行点灯命令

```
void setup() {  
  // put your setup code here, to run once:  
  // 设置引脚为输出模式  
  pinMode(led_pin, OUTPUT);  
  // 点亮LED，给引脚赋值一个高电平的值  
  digitalWrite(led_pin, HIGH);  
}  
  
void loop() {  
  // put your main code here, to run repeatedly:  
}
```

1.2、效果图



15°C 晴明

14:15 2025/11/24

(PS:一开始我把两轮的代码分别放在了两个仓库里，后面又新建了一个仓库——iot-recruitment-ChenXingyu，并且创建了一个分支——StarsofUESTC-patch-1(IoT-Dirction))

二、并发思维

1、非阻塞编程

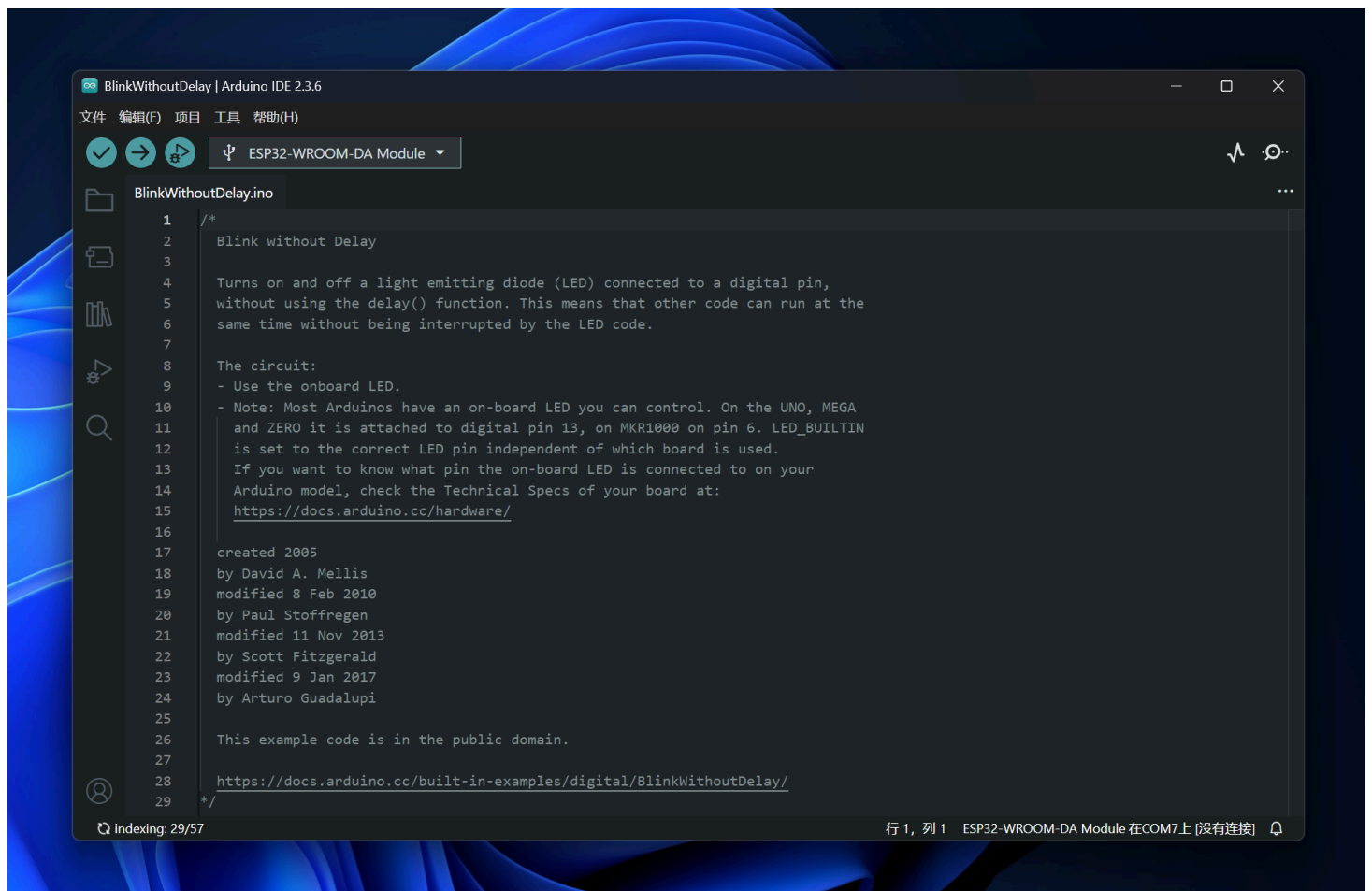
millis()函数

1.1、它是Arduino内置的一个函数，它返回从开发板上电或复位开始到现在所经过的毫秒数

1.2、该毫秒数由一个硬件定时器在后台自动累计，完全独立于主程序

1.3、“升级版”：micros()函数，微秒数

学习官方



英语四级阅读ing

关键信息：

- 1、设置LED引脚的编号为常量，设置LED状态为变量
- 2、使用unsigned long数据类型而不是int数据类型： unsigned long——32位整数，int——16位整数，很快会溢出

2、代码重构——学习一个新的电子模块

定时读取传感器：每隔2秒钟，自动读取一次DHT11的温湿度数据。

定时打印数据：每隔5秒钟，将最新的温湿度数据打印到串口监视器上。

接云平台 —— 让数据“活”起来

器只能在局域网内访问，且功能有限。现在，我们把数据发送到专业的物联网云平台。

云平台：

用对新手最友好的ThingSpeak平台。

个账号，创建一个Channel，并获取你的Write API Key。

的代码上，增加一个新功能：每隔30秒，通过HTTP GET请求，将你的温湿度数据发送到ThingSpeak的Channel中。

你将看到由你的设备上传的数据所形成的实时图表。

思维)：

新的网页路径，例如 /json？当访问 /json 时，而是返回一段JSON格式的字符串。

```
"temp": 25.5, "humidity": 60, "led_status": "ON"}
```

学习简单的字符串拼接，并了解什么是JSON。推荐使用ArduinoJson

差点以为买到的DHT11是次品，因为在B站上查了一下，发现其他的DHT11都略大一些，并且有明显的引脚标记。后面了解到浅蓝色的部分就是DHT11的主体，其他部分是焊接上去的，类似于注释，问了问商家，还是有说明：

话说在那个学习资源清单的文件里发现了额外的要求——将变量(温湿度)显示在网页上，差点漏掉

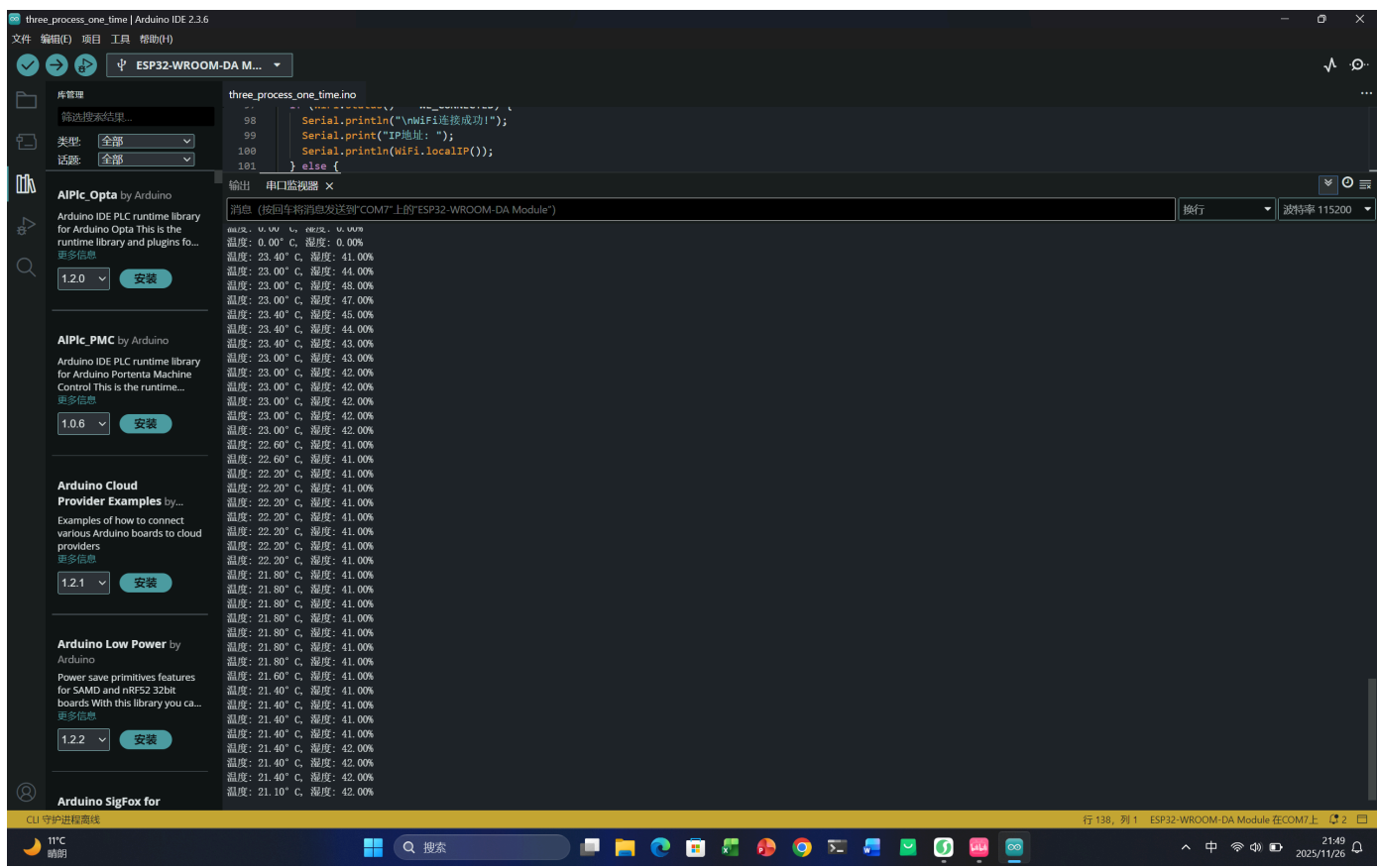
2. 在网页上显示数据 (升级界面):

- 任务： 将变量（温湿度）显示在你的网页上。
- 核心关键词：
 - Arduino String字符串拼接 (你需要把HTML代码和变量数据组合成一个长字符串)

■ ESP32 网页显示传感器数据

- 核心思路： 在第一期的基础上，先读取温湿度存为变量，然后在使用 client.println() 发送网页代码时，把这些变量也一起发送出去。

这里先将数据打印在串口通信处：



电路连接示意图：



如何得到多样的数据?(嘿嘿)

温度: 25.30° C, 湿度: 43.00%
温度: 24.80° C, 湿度: 43.00%
温度: 24.80° C, 湿度: 42.00%
温度: 24.80° C, 湿度: 54.00%
温度: 25.60° C, 湿度: 72.00%
温度: 26.20° C, 湿度: 81.00%
温度: 27.60° C, 湿度: 89.00%
温度: 28.00° C, 湿度: 87.00%
温度: 28.90° C, 湿度: 85.00%
温度: 29.30° C, 湿度: 85.00%
温度: 31.80° C, 湿度: 98.00%
温度: 32.80° C, 湿度: 98.00%
温度: 32.30° C, 湿度: 98.00%
温度: 31.80° C, 湿度: 98.00%
温度: 31.30° C, 湿度: 91.00%
温度: 31.30° C, 湿度: 80.00%
温度: 30.80° C, 湿度: 67.00%
温度: 30.80° C, 湿度: 59.00%
温度: 30.20° C, 湿度: 50.00%
温度: 30.20° C, 湿度: 47.00%
温度: 29.80° C, 湿度: 45.00%
温度: 29.30° C, 湿度: 43.00%
温度: 28.90° C, 湿度: 42.00%
温度: 28.90° C, 湿度: 41.00%
温度: 28.50° C, 湿度: 40.00%
温度: 28.00° C, 湿度: 39.00%
温度: 28.00° C, 湿度: 38.00%
温度: 27.60° C, 湿度: 39.00%
温度: 27.60° C, 湿度: 39.00%
温度: 27.10° C, 湿度: 39.00%
温度: 26.70° C, 湿度: 39.00%
温度: 26.70° C, 湿度: 39.00%
温度: 26.20° C, 湿度: 40.00%
温度: 26.20° C, 湿度: 39.00%
温度: 26.20° C, 湿度: 39.00%
温度: 26.20° C, 湿度: 39.00%

把DHT11握在手里/哈一口气，然后发现温度和湿度都迅速上升，松开后，数值又逐渐下降

三、连接云平台

1、注册与学习(已经能非常熟练地科学上网了)

这里创建new channel的时候，遇到许多需要填写的内容，但感觉右侧的英文没太看懂，就问问了AI

ThingSpeak™ Channels Apps Support

Commercial Use How to Buy

To use ThingSpeak, you must sign in with your existing MathWorks account or create a new one.

Non-commercial users may use ThingSpeak for free. Free accounts offer limits on certain functionality. Commercial users are eligible for a time-limited free evaluation. To get full access to the MATLAB analysis features on ThingSpeak, log in to ThingSpeak using the email address associated with your university or organization.

To send data faster to ThingSpeak or to send more data from more devices, consider the [paid license options](#) for commercial, academic, home and student usage.

MathWorks®

← 3568969688@qq.com

Password

Forgot Password?

Sign In

SMART CONNECTED DEVICES

DATA AGGREGATION AND ANALYTICS
ThingSpeak™

MATLAB®

ALGORITHM DEVELOPMENT
SENSOR ANALYTICS

Blog | Documentation | Tutorials | Terms | Privacy Policy

© 2025 The MathWorks, Inc.

New Channel

Name

Description

Field 1 ☒

Field 2 ☐

Field 3 ☐

Field 4 ☐

Field 5 ☐

Field 6 ☐

Field 7 ☐

Field 8 ☐

Metadata

Tags
(Tags are comma separated)

Link to External Site

Link to GitHub

Help

Channels store all the data that a ThingSpeak application collects. Each channel includes eight fields that can hold any type of data, plus three fields for location data and one for status data. Once you collect data in a channel, you can use ThingSpeak apps to analyze and visualize it.

Channel Settings

- **Percentage complete:** Calculated based on data entered into the various fields of a channel. Enter the name, description, location, URL, video, and tags to complete your channel.
- **Channel Name:** Enter a unique name for the ThingSpeak channel.
- **Description:** Enter a description of the ThingSpeak channel.
- **Field#:** Check the box to enable the field, and enter a field name. Each ThingSpeak channel can have up to 8 fields.
- **Metadata:** Enter information about channel data, including JSON, XML, or CSV data.
- **Tags:** Enter keywords that identify the channel. Separate tags with commas.
- **Link to External Site:** If you have a website that contains information about your ThingSpeak channel, specify the URL.
- **Show Channel Location:**
 - **Latitude:** Specify the latitude position in decimal degrees. For example, the latitude of the city of London is 51.5072.
 - **Longitude:** Specify the longitude position in decimal degrees. For example, the longitude of the city of London is -0.1275.
 - **Elevation:** Specify the elevation position meters. For example, the elevation of the city of London is 35.052.
- **Video URL:** If you have a YouTube™ or Vimeo® video that displays your channel information, specify the full path of the video URL.
- **Link to GitHub:** If you store your ThingSpeak code on GitHub®, specify the GitHub repository URL.

Using the Channel

You can get data into a channel from a device, website, or another ThingSpeak channel. You can then visualize data and transform it using ThingSpeak [Apps](#).

Field 7 ☐

Field 8 ☐

Metadata

Tags
(Tags are comma separated)

Link to External Site

Link to GitHub

Elevation

Show Channel Location ☐

Latitude

Longitude

Show Video ☐

☒ YouTube

☐ Vimeo

Video URL

Show Status ☐

- **Show Channel Location:**
 - **Latitude:** Specify the latitude position in decimal degrees. For example, the latitude of the city of London is 51.5072.
 - **Longitude:** Specify the longitude position in decimal degrees. For example, the longitude of the city of London is -0.1275.
 - **Elevation:** Specify the elevation position meters. For example, the elevation of the city of London is 35.052.
- **Video URL:** If you have a YouTube™ or Vimeo® video that displays your channel information, specify the full path of the video URL.
- **Link to GitHub:** If you store your ThingSpeak code on GitHub®, specify the GitHub repository URL.

Using the Channel

You can get data into a channel from a device, website, or another ThingSpeak channel. You can then visualize data and transform it using ThingSpeak [Apps](#).

See [Get Started with ThingSpeak™](#) for an example of measuring dew point from a weather station that acquires data from an Arduino® device.

[Learn More](#)

也是顺利创建了一个Channel了 😊

ThingSpeak™

Channels Apps Devices Support

Commercial UseHow to Buy

Signed in successfully.

My Channels

New Channel

Search by tag

Name	Created	Updated
ESP32温湿度检测 esp32, dht11, iot PrivatePublicSettingsSharingAPI KeysData Import / Export	2025-11-27	2025-11-27 00:08

Help

Collect data in a ThingSpeak channel from a device, from another channel, or from the web.

Click **New Channel** to create a new ThingSpeak channel.

Click on the column headers of the table to sort by the entries in that column or click on a tag to show channels with that tag.

Learn to **create channels**, explore and transform data.

Learn more about **ThingSpeak Channels**.

Examples

- Arduino
- Arduino MKR1000
- ESP8266
- Raspberry Pi
- Netduino Plus

Upgrade

Need to send more data faster?

Need to use ThingSpeak for a commercial project?

Upgrade

Blog | Documentation | Tutorials | Terms | Privacy Policy



© 2025The MathWorks, Inc.

ThingSpeak™

Channels Apps Devices Support

Commercial UseHow to Buy

ESP32温湿度检测

Channel ID: 3183330

Author: mwa0000039748039

Access: Private

通过ESP32, 使用DHT11传感器来采集温湿度数据

esp32, dht11, iot

Private ViewPublic ViewChannel Settings

SharingAPI KeysData Import / Export

Channel Sharing Settings

☐ Keep channel view private

☒ Share channel view with everyone

☐ Share channel view only with the following users:

Email Address

Enter email here

Add User

Help

ThingSpeak allows you to control who can view the data in your channel. Irrespective of the settings on this tab, reading data from or writing data to the fields of a channel requires the appropriate API key for the channel.

Channel Sharing Settings

- Keep channel view private:** Selecting this option keeps your channel private. Only you will be able to see the channel view.
- Share channel view with everyone:** Selecting this option makes the public view of your channel viewable by anyone browsing the ThingSpeak website.
- Share channel view only with the following users:** Selecting this option shares the private view of your channel only with specific ThingSpeak users.

Blog | Documentation | Tutorials | Terms | Privacy Policy



© 2025The MathWorks, Inc.

2、数据上云

- 什么是HTTP GET请求？
- 1、HTTP：超文本传输协议(unsafe)

2、GET：HTTP协议中的一种请求方法，用于"获取"数据

3、即用于提取数据，如查看网页、读取传感器数据

4、本题中用于将ESP32采集的温湿度数据通过URL参数的形式发送到ThingSpeak云平台，实现数据的远程存储和可视化。

新增的这个功能，和之前的LED、DHT11一样，需要：

1、导入<HttpClient.h>库

2、为了后续构建URL，先配置Thingspeak,包括API密钥和thingspeak官网名

3、定义好thingspeak的初始时间和时间间隔

4、接下来，是最关键的数据上云函数——data_to_thingspeak()

4.1、首先，在if语句判断wifi连接成功的前提下，发送HTTP请求

4.2、构建thingspeak API的URL,顺序依次为：HTTP协议开头、thingspeak的官方网址(前面已经配置好)、它的数据更新接口与API密钥参数、温度数据、湿度数据

4.3、终于，构建好URL后，可以发送HTTP GET请求

4.4、问了问AI，它帮我补充了这两个数据上传结果处理与反馈部分，在实际使用场景中可以帮助我们快速知道系统运行的状态

4.5、最后，关闭HTTP连接并释放内存！

5、在loop函数中，同样使用if语句，来实现数据每隔30s上传一次

关于页面的前端设计部分，我是在网上查的，也借助了AI

最终结果：

23:44



4G 90

ESP32智能监控系统

实时传感器数据

温度: 21.4°C

湿度: 60.0%

LED远程控制

开灯

关灯

状态: ☐ 关闭

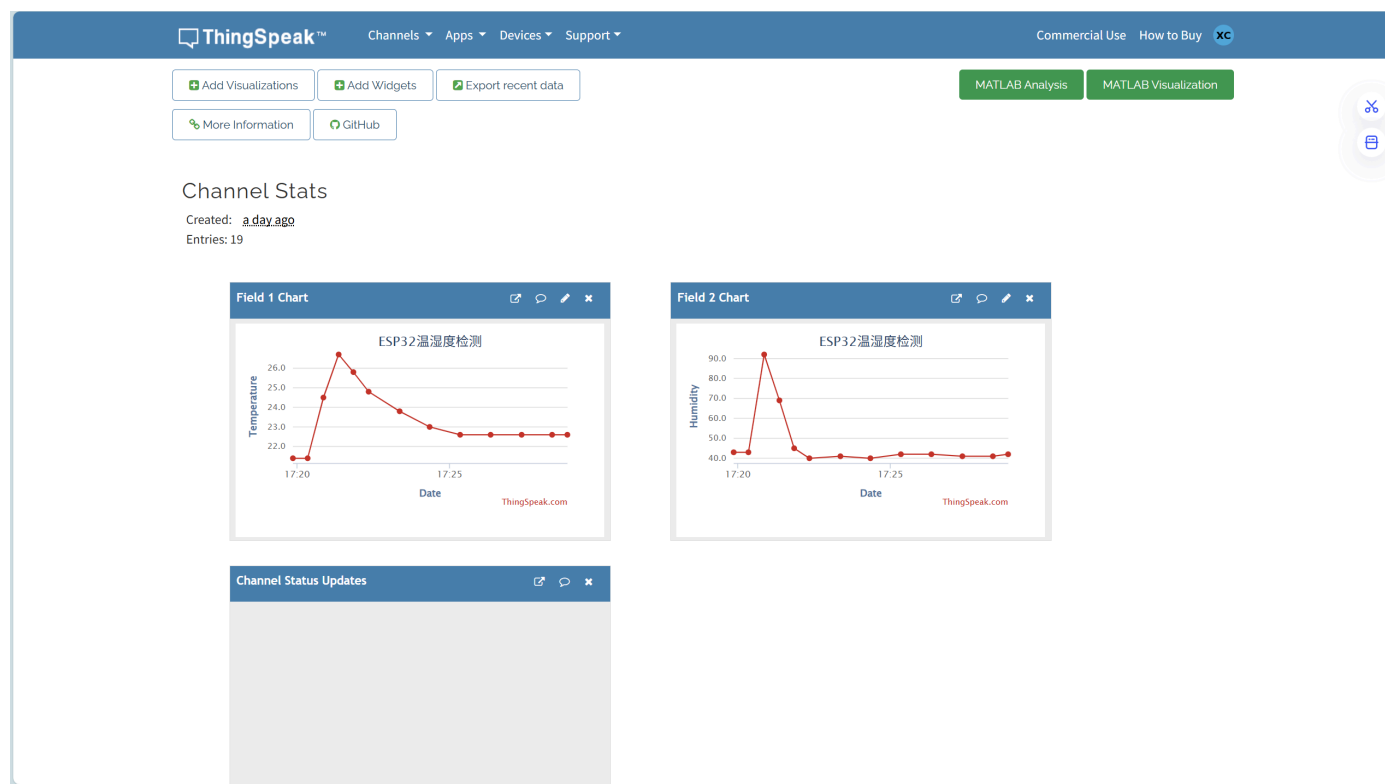
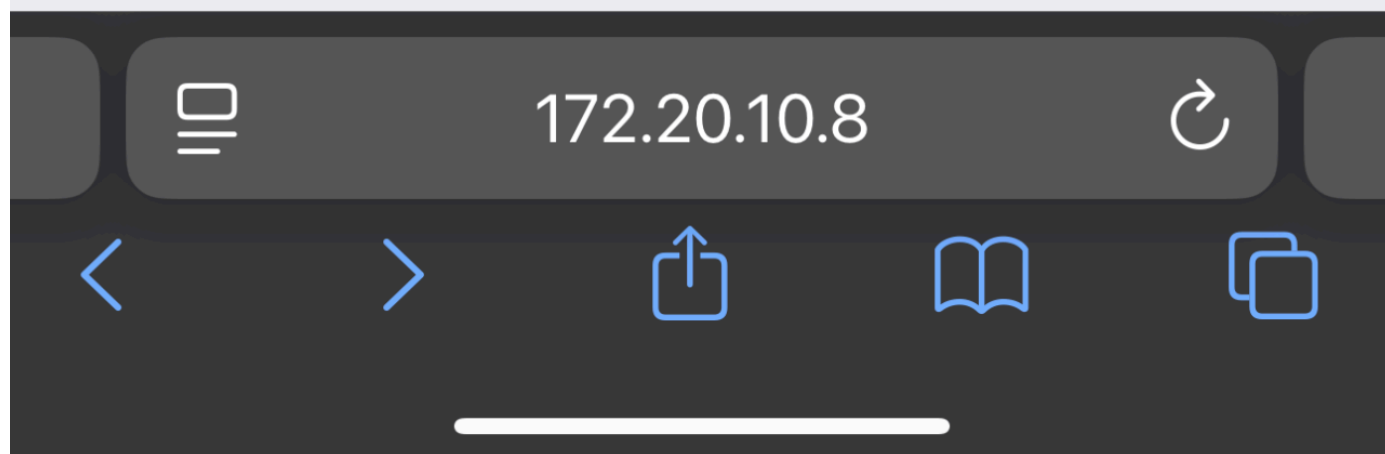
REST API接口

获取JSON格式数据:

访问 /json API

返回格式: {"temperature":25.5, "humidity":60.2,
"led_status":"ON"}

API地址: http://172.20.10.8/json



四、API思维

一个新的网页路径——/json

安装ArduinoJson库

本来我想自己是新手，可能手动拼接的方法更适合我，结果AI建议我新手就是要挑战用ArduinoJson库，那就来吧！



在原来的代码基础上进行修改：

- 1、引用ArduinoJson库——<ArduinoJson.h>
- 2、创建Json函数：

```

void Json(){
    // 创建动态JSON文档,灵活地添加各种数据类型
    DynamicJsonDocument doc(1024);//可以存储1024字节

    // 添加数据到JSON对象
    doc["temperature"] = temperature;
    doc["humidity"] = humidity;
    doc["led_status"] = ledState ? "ON" : "OFF";//三元运算符,True为ON, False为OFF
    doc["timestamp"] = millis();
    doc["device"] = "ESP32";//设备标识字段
    doc["ip_address"] = WiFi.localIP().toString();//获取ESP32的IP地址,将IP地址转换为字符串格式

    // 序列化为字符串
    String json;//创建空字符串用于存储序列化结果
    serializeJson(doc, json);//将JSON文档转换为字符串格式

    // 发送JSON响应
    server.send(200, "application/json", json);
    Serial.println("JSON API被访问,数据已发送");
}

```

3、进行字符串拼接并借助AI实现前端的页面

```

// API接口说明区域
html += "<div class='container'>";
html += "<h2> 🌟 REST API接口</h2>";
html += "<div class='api-section'>";
html += "<p>获取JSON格式数据:</p>";
html += "<a href='/json' class='api-btn'>访问 /json API</a>";
html += "<p><small>返回格式: {\"temperature\":25.5, \"humidity\":60.2, \"led_status\": \"ON\"}</small></p>";
html += "</div>";
html += "<p><small>API地址: http://\" + WiFi.localIP().toString() + \"/json</small></p>";
html += "</div>";

html += "</body></html>";

server.send(200, "text/html", html);

```

手机页面:

23:44



4G 90

ESP32智能监控系统

实时传感器数据

温度: 21.4°C

湿度: 60.0%

LED远程控制

开灯

关灯

状态: ☐ 关闭

REST API接口

获取JSON格式数据:

访问 /json API

返回格式: {"temperature":25.5, "humidity":60.2,
"led_status":"ON"}

API地址: <http://172.20.10.8/json>



172.20.10.8



23:44

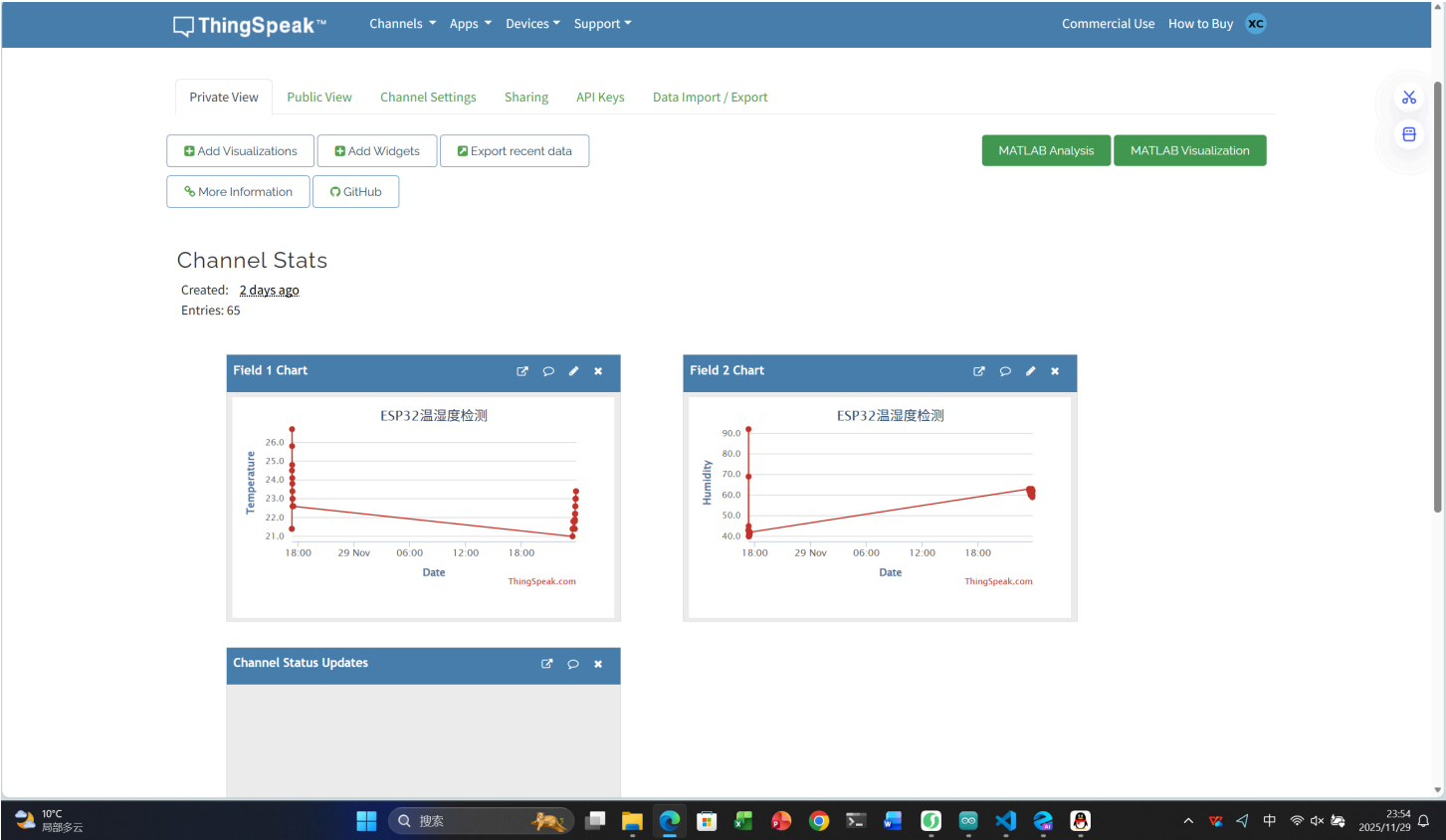


4G

90

```
{"temperature":21.4,"humidity":60,"led_status":"OFF",  
"timestamp":38265,"device":"ESP32","ip_address":"172.  
20.10.8"}
```


再来一张



OK，那招新题到这里就先告一段落啦 😊,虽然总共加起来题目页数还没有两位数，但确实感觉自己在11月中关于嵌入式IoT学到了许多，以后继续加油！